

JAVA DOCS

The Java Collections Framework

List, Set, Map, Queue — Interfaces, Implementations & Internals

Java Agentic AI — Knowledge Base Reference

1. Overview

The Java Collections Framework (JCF), introduced in Java 1.2, is a unified architecture for storing and manipulating groups of objects. It provides interfaces (abstract data types), implementations (concrete classes), and algorithms (sorting, searching via the Collections utility class).

Root Interface	Sub-interfaces / Key Implementations
Collection	List, Set, Queue
List	ArrayList, LinkedList, Vector, Stack
Set	HashSet, LinkedHashSet, TreeSet
Queue / Deque	ArrayDeque, LinkedList, PriorityQueue
Map (separate hierarchy)	HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap

Note: Map does NOT extend Collection — it's a separate root interface because it stores key-value pairs, not single elements. This is a classic interview trap.

2. List Interface — Ordered, Duplicates Allowed

Implementation	Backing structure	Access	Insert/Delete (middle)	Thread-safe
ArrayList	Dynamic array	O(1)	O(n) — shifts elements	No
LinkedList	Doubly linked list	O(n)	O(1) once positioned	No
Vector	Dynamic array	O(1)	O(n)	Yes (synchronized, legacy)
Stack	Extends Vector (LIFO)	O(1) top	O(1) push/pop	Yes (legacy)

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add(0, "Z");           // insert at index 0
list.get(1);                // "A"
list.remove("B");           // remove by value
list.remove(0);             // remove by index
Collections.sort(list);     // natural ordering
list.sort(Comparator.reverseOrder());

List<Integer> immutable = List.of(1, 2, 3); // Java 9+ immutable list factory
// immutable.add(4);                       // throws UnsupportedOperationException
```

Note: ArrayList grows by allocating a new array (typically 1.5x capacity) and copying elements when full — this makes amortized append O(1) but an occasional resize is O(n). LinkedList avoids resizing but has higher per-element memory overhead (node objects with prev/next pointers) and poor cache locality.

3. Set Interface — No Duplicates

Implementation	Ordering	Backed by	Null allowed
HashSet	No guaranteed order	HashMap internally	One null
LinkedHashSet	Insertion order preserved	HashMap + linked list	One null
TreeSet	Sorted (natural or Comparator)	Red-Black tree (TreeMap)	No (throws NPE)

```
Set<String> set = new HashSet<>();
set.add("apple"); set.add("banana"); set.add("apple"); // duplicate ignored
System.out.println(set.size()); // 2

TreeSet<Integer> sorted = new TreeSet<>(List.of(5, 1, 3));
System.out.println(sorted); // [1, 3, 5]
sorted.first(); sorted.last();
sorted.higher(3); // smallest element strictly greater than 3
sorted.floor(4); // largest element <= 4
```

Note: *HashSet uses the element's hashCode() to bucket it and equals() to detect duplicates — both must be correctly overridden for custom objects, or duplicates will silently slip through.*

4. Map Interface — Key-Value Pairs

Implementation	Ordering	Null key	Thread-safe
HashMap	No guaranteed order	One null key allowed	No
LinkedHashMap	Insertion (or access) order	One null key allowed	No
TreeMap	Sorted by key	No (NPE)	No
Hashtable	No guaranteed order	No null key/value	Yes (legacy, coarse lock)
ConcurrentHashMap	No guaranteed order	No null key/value	Yes (segment/bucket-level locking)

```
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
map.put("b", 2);
map.putIfAbsent("a", 99); // no-op, key exists
map.getOrDefault("z", 0); // 0 - default if missing
map.computeIfAbsent("c", k -> 3); // adds "c" -> 3
map.merge("a", 10, Integer::sum); // "a" -> 1 + 10 = 11

for (Map.Entry<String, Integer> e : map.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}

map.forEach((k, v) -> System.out.println(k + ":" + v));
```

4.1 How HashMap Works Internally

A HashMap stores entries in an array of **buckets**. On put(key, value): (1) compute hash(key.hashCode()), applying a supplemental hash spreading function to reduce collisions; (2) find the bucket index via hash & (capacity - 1); (3) if the bucket is empty, insert a new node; if occupied, walk the

chain comparing hashCode+equals, replacing on a match or appending otherwise. Since Java 8, when a single bucket's chain exceeds 8 entries (and the table has at least 64 buckets), that chain is converted from a linked list to a **red-black tree**, bounding worst-case lookup at $O(\log n)$ instead of $O(n)$. The map resizes (doubles capacity, rehashes all entries) once size exceeds capacity * loadFactor (default load factor 0.75).

Operation	Average case	Worst case (pre-Java 8 / heavy collisions)
get / put / remove	$O(1)$	$O(n)$ — or $O(\log n)$ with Java 8+ treeified buckets
containsKey	$O(1)$	$O(\log n)$ treeified

5. Queue and Deque

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(1); queue.offer(2);
queue.poll();           // removes & returns head (1) or null if empty
queue.peek();           // views head without removing

Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1); deque.addLast(2);
deque.pollFirst(); deque.pollLast();

// PriorityQueue - min-heap by default
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5); pq.offer(1); pq.offer(3);
pq.poll(); // 1 (smallest first)

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
```

Note: *ArrayDeque is generally preferred over LinkedList for both stack and queue use cases today — it's more memory-efficient (no per-node object overhead) and has better cache locality.*

6. Iterating & Fail-Fast Behavior

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4));
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int val = it.next();
    if (val == 2) it.remove(); // safe removal during iteration
}

for (int val : list) {
    if (val == 3) list.remove(Integer.valueOf(3)); // throws
    ConcurrentModificationException!
}
```

Note: *Most JCF collections are **fail-fast**: they maintain a modCount and throw ConcurrentModificationException if the collection is structurally modified during iteration by any means other than the iterator's own remove(). CopyOnWriteArrayList and ConcurrentHashMap are **fail-safe** (weakly consistent) instead — they iterate over a snapshot or tolerate concurrent structural changes without throwing.*

7. Comparable vs Comparator

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Location	Implemented inside the class itself	Separate class, or lambda passed externally
Number of orderings	Exactly one 'natural ordering'	Unlimited — define as many as needed

```
class Employee implements Comparable<Employee> {
    String name; int age;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.age, other.age); // natural order = by age
    }
}

List<Employee> employees = ...;
Collections.sort(employees); // uses compareTo

employees.sort(Comparator.comparing(e -> e.name)); // by name
employees.sort(Comparator.comparing((Employee e) -> e.age).reversed());
employees.sort(Comparator.comparing((Employee e) -> e.name)
    .thenComparing(e -> e.age)); // multi-level sort
```

8. Generics in Collections (Quick Reference)

The Generics document covers this topic in full depth. In collections, generics provide compile-time type safety, eliminating manual casting and `ClassCastException` risks that plagued pre-Java-5 raw-type collections.

```
List rawList = new ArrayList(); // raw type - avoid, no type safety
List<String> typed = new ArrayList<>(); // type-safe

List<? extends Number> readOnly = List.of(1, 2.0, 3L); // upper bounded wildcard - producer
List<? super Integer> writable = new ArrayList<Number>(); // lower bounded wildcard - consumer
```

9. The Collections and Arrays Utility Classes

```
Collections.sort(list);
Collections.reverse(list);
Collections.shuffle(list);
Collections.max(list); Collections.min(list);
Collections.unmodifiableList(list); // read-only wrapper (view, not a deep copy)
Collections.synchronizedList(list); // synchronized wrapper for thread safety
Collections.emptyList();

int[] arr = {5, 3, 1};
```

```
Arrays.sort(arr);
List<Integer> asList = Arrays.asList(1, 2, 3); // fixed-size view backed by the array
```

Note: `Collections.unmodifiableList()` returns a view — if the underlying original list is mutated directly, that change IS visible through the 'unmodifiable' wrapper. For a true immutable snapshot, use `List.copyOf(list)` or `List.of(...)` instead.

9b. Thread-Safe & Concurrent Collections

Beyond legacy synchronized wrappers, `java.util.concurrent` provides purpose-built concurrent collections that scale far better under contention.

Class	Use case	Concurrency strategy
<code>ConcurrentHashMap</code>	General-purpose concurrent map	Bucket-level locking / CAS, no full-map lock
<code>CopyOnWriteArrayList</code>	Read-heavy, rarely-written lists	Copies the whole array on every write
<code>BlockingQueue</code> (impls)	Producer-consumer pipelines	Blocks on empty (take) / full (put) queue
<code>ConcurrentSkipListMap</code>	Concurrent sorted map	Lock-free skip list

```
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);
queue.put(task);           // blocks if queue is full
Task t = queue.take();     // blocks if queue is empty

ConcurrentHashMap<String, Integer> chm = new ConcurrentHashMap<>();
chm.compute("key", (k, v) -> (v == null) ? 1 : v + 1); // atomic read-modify-write
```

9c. NavigableMap and NavigableSet

`TreeMap` and `TreeSet` implement `NavigableMap`/`NavigableSet`, exposing rich range-query operations beyond a plain sorted structure.

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(1, "a"); map.put(5, "b"); map.put(10, "c");

map.floorKey(6);    // 5 - largest key <= 6
map.ceilingKey(6);  // 10 - smallest key >= 6
map.firstKey();     // 1
map.lastKey();      // 10
map.headMap(5);     // keys strictly less than 5 -> {1=a}
map.tailMap(5);     // keys >= 5 -> {5=b, 10=c}
map.descendingMap(); // reverse-ordered view
```

9d. Complexity Cheat Sheet

Structure	get/contains	add	remove
<code>ArrayList</code>	$O(1)$ by index / $O(n)$ by value	$O(1)$ amortized (end)	$O(n)$

Structure	get/contains	add	remove
LinkedList	$O(n)$	$O(1)$ at ends	$O(1)$ once positioned
HashMap / HashSet	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
TreeMap / TreeSet	$O(\log n)$	$O(\log n)$	$O(\log n)$
ArrayDeque	$O(1)$ at ends	$O(1)$ at ends	$O(1)$ at ends
PriorityQueue	$O(1)$ peek min/max	$O(\log n)$	$O(\log n)$

10. Quick Interview Recap

- ArrayList vs LinkedList? ArrayList: fast random access $O(1)$, slow middle insert/delete $O(n)$.
LinkedList: slow random access $O(n)$, fast insert/delete once positioned $O(1)$.
- HashMap vs Hashtable vs ConcurrentHashMap? HashMap is unsynchronized and allows one null key; Hashtable is legacy, fully synchronized (whole-map lock), no nulls; ConcurrentHashMap uses fine-grained locking/CAS for much better concurrent throughput, also no nulls.
- How does HashSet detect duplicates? Via the element's hashCode() (bucket selection) followed by equals() (exact match check) within that bucket.
- What is load factor? The threshold (default 0.75) of fill ratio at which a HashMap resizes (doubles capacity) and rehashes all entries to keep operations close to $O(1)$.
- Fail-fast vs fail-safe iterators? Fail-fast throws ConcurrentModificationException on detected structural changes (most JCF collections); fail-safe iterates a snapshot or tolerates changes (CopyOnWriteArrayList, ConcurrentHashMap).